1. Objetivo

Reimplementar a função timer_sleep() do sistema operacional educacional Pintos, substituindo a espera ocupada (busy waiting) por um mecanismo de bloqueio e despertar de threads. A implementação calcula o instante de despertar da thread, adiciona a thread a uma lista de threads dormindo e a bloqueia durante a espera. A thread é posteriormente desbloqueada pelo tratamento da interrupção do timer quando o tempo necessário é atingido.

2. Checklist de Requisitos

Requisito do enunciado	Status	Como testar / evidência
Reimplementar timer_sleep()	OK	Compilar o projeto e executar os testes de Alarm Clock. A implementação está em src/devices/timer.c.
Evitar *busy waiting* em timer_sleep()	OK	Inspecionar timer_sleep(): a thread é adicionada à lista de espera e chamada thread_block() em vez de permanecer executando e chamando thread_yield() repetidamente.
Calcular o instante em que a thread deve acordar	OK	timer_sleep() define current->wake_up_tick = timer_ticks() + ticks.
Manter as threads que estão dormindo em uma estrutura de espera	OK	timer.c possui sleeping_list e adiciona a thread por meio de list_push_back().
Bloquear a thread enquanto ela aguarda	OK	timer_sleep() chama thread_block() após inserir a thread na lista.
Verificar as threads que devem acordar durante a interrupção do timer	OK	timer_interrupt() percorre sleeping_list e verifica wake_up_tick <= ticks.
Desbloquear a thread quando o tempo for atingido	OK	Quando o tempo é atingido, o elemento é removido da lista e thread_unblock() é chamado.
alarm-single	OK	build/tests/threads/alarm-single.result retornou PASS.

Requisito do enunciado	Status	Como testar / evidência
alarm-multiple	OK	build/tests/threads/alarm-multiple.result retornou PASS.
alarm-simultaneous	OK	build/tests/threads/alarm-simultaneous.result retornou PASS.
alarm-zero	OK	build/tests/threads/alarm-zero.result retornou PASS.
alarm-negative	OK	build/tests/threads/alarm-negative.result retornou PASS.

3. Como Reproduzir

Compilar

A partir do diretório src/threads:
cd ~/InfrSW/pintos/src/threads
make

Executar o Pintos

pintos -- -q

Executar os testes

make check

Os resultados dos testes de Alarm Clock podem ser verificados individualmente em:

build/tests/threads/

Para verificar os cinco testes:

cat build/tests/threads/alarm-single.result
cat build/tests/threads/alarm-multiple.result
cat build/tests/threads/alarm-simultaneous.result
cat build/tests/threads/alarm-zero.result
cat build/tests/threads/alarm-negative.result

Resultado esperado para cada um:

PASS

Durante a sessão registrada no evidencias.log, o make check foi interrompido manualmente durante a execução de mlfqs-load-avg. Por isso, a validação final dos requisitos de Alarm Clock foi feita também por meio dos resultados individuais dos cinco testes específicos.

4. Arquitetura

Arquivos e responsabilidades

1. src/devices/timer.c → Implementa a lista de threads dormindo, o novo comportamento de timer_sleep() e o despertar das threads durante a interrupção do timer.
2. src/threads/thread.h → Armazena na estrutura struct thread os dados necessários para controlar o despertar da thread, incluindo wake_up_tick e sleep_elem.
3. src/threads/thread.c → Fornece as estruturas e funções de gerenciamento de threads utilizadas pelo mecanismo de bloqueio e desbloqueio.

Decisões técnicas

1. Usar uma lista de threads dormindo → Permitir que várias threads aguardem simultaneamente seus respectivos tempos → Cada thread permanece registrada até atingir seu instante de despertar.
2. Armazenar o instante absoluto de despertar em wake_up_tick → Comparar diretamente o tempo de despertar com o contador global de ticks → A interrupção do timer consegue identificar quais threads já podem ser desbloqueadas.
3. Bloquear a thread com thread_block() durante a espera → Evitar que a thread continue consumindo CPU enquanto espera → O processador pode executar outras threads até que a thread seja desbloqueada.
4. Realizar o desbloqueio no timer_interrupt() → Aproveitar a interrupção periódica do timer para verificar os tempos de despertar → As threads são colocadas novamente no estado de prontas quando o tempo é atingido.
5. Desabilitar interrupções durante a alteração da lista de espera → Evitar inconsistências enquanto a thread é inserida na lista e bloqueada → A operação de espera fica protegida contra uma interrupção concorrente nesse trecho.

5. Estratégias e Diário de Desenvolvimento

5.1 Estratégias da semana

Uma linha por abordagem diferente que você tentou. Se você não mudou de abordagem, preencha só S1.

Estratégia	Nome curto	Contexto	Motivo da troca (se houve)
S1	Validação inicial	Verificação do ambiente, compilação, execução do Pintos e tentativa de executar a suíte geral de testes.	A execução de make check foi interrompida durante um teste que não estava relacionado diretamente aos cinco testes básicos de Alarm Clock.
S2	Testes específicos	Busca e verificação individual dos arquivos de resultado dos testes alarm-*.	Não houve nova troca de abordagem registrada.

5.2 Diário de Tentativas

Uma linha por tentativa relevante, marcando a qual estratégia (S1/S2/S3) ela pertence, obviamente precisa ser em ordem cronológica. A coluna "Quando" precisa bater com a ordem cronológica do evidencias.log, é isso que comprova que o fluxo é real. "Hipótese/Causa" só é preenchida quando o resultado foi falha ou parcial.

#	Estrat.	O que tentei	Resultado	Hipótese/Causa (se falhou)	Quando
1	S1	Registrei date, whoami e pwd no início da sessão.	OK	OK	19:13:38
2	S1	Executei make em src/threads.	OK		após 19:13:38
3	S1	Executei pintos -- -q.	OK		após a compilação
4	S1	Executei make check.	Parcial	A execução foi interrompida manualmente durante o teste mlfqs-load-avg.	após a execução do Pintos
5	S2	Tentei executar os testes de Alarm Clock usando os caminhos tests/threads/*.output.	Falha	Os arquivos .output não eram alvos diretamente disponíveis a partir daquele diretório pelo comando utilizado.	após make check
6	S2	Executei find . -name '*alarm*' para localizar os arquivos existentes.	OK		após a falha anterior
7	S2	Verifiquei alarm-single.result.	OK		após a localização dos arquivos

#	Estrat.	O que tentei	Resultado	Hipótese/Causa (se falhou)	Quando
8	S2	Verifiquei alarm-multiple.result.	OK		após a tentativa anterior
9	S2	Verifiquei alarm-simultaneous.result.	OK		após a tentativa anterior
10	S2	Verifiquei alarm-zero.result.	OK		após a tentativa anterior
11	S2	Verifiquei alarm-negative.result.	OK		após a tentativa anterior
12	S2	Encerrei a sessão com exit.	OK		19:20:23

6. Evidências

6.1 Log automático

Foi utilizado o comando:

script -a evidencias.log

O log registra a sessão de desenvolvimento iniciada em:

2026-09-21 19:13:13 -03:00

e encerrada em:

2026-09-21 19:20:23 -03:00

O início da sessão também registra:

date
Mon Sep 21 19:13:38 -03 2026

whoami
danda

pwd
/home/danda/InfrSW/pintos

O arquivo evidencias.log acompanha a entrega dentro do .tar.

6.2 Prints

Erro principal

```
danda@LAPTOP-CDROKK7L:~/InfrSW/pintos/src/threads$ make tests/threads/alarm-single.output tests/thread
s/alarm-multiple.output tests/threads/alarm-simultaneous.output tests/threads/alarm-zero.output tests/
threads/alarm-negative.output
make: *** No rule to make target 'tests/threads/alarm-single.output'.  Stop.
```

Validação final

```
danda@LAPTOP-CDROKK7L:~/InfrSW/pintos/src/threads$ cat build/tests/threads/alarm-single.result
PASS
danda@LAPTOP-CDROKK7L:~/InfrSW/pintos/src/threads$ cat build/tests/threads/alarm-multiple.result
PASS
danda@LAPTOP-CDROKK7L:~/InfrSW/pintos/src/threads$ cat build/tests/threads/alarm-multiple.result
PASS
danda@LAPTOP-CDROKK7L:~/InfrSW/pintos/src/threads$ cat build/tests/threads/alarm-simultaneous.result
PASS
danda@LAPTOP-CDROKK7L:~/InfrSW/pintos/src/threads$ cat build/tests/threads/alarm-zero.result
PASS
danda@LAPTOP-CDROKK7L:~/InfrSW/pintos/src/threads$ cat build/tests/threads/alarm-negative.result
PASS
```

7. Uso de IA

Onde usei IA

Foi utilizada inteligência artificial como apoio durante o desenvolvimento para interpretar erros do ambiente de execução, auxiliar na compreensão dos comandos do Pintos/WSL e organizar a documentação da atividade.

Prompts principais

- Solicitação de ajuda para configurar e executar o ambiente do Pintos no WSL.
- Solicitação de ajuda para interpretar erros de compilação e execução dos testes.
- Solicitação de ajuda para organizar o relatório e as evidências da atividade.

O que validei manualmente e como

A implementação foi validada manualmente por meio da compilação com make, da execução do Pintos com pintos -- -q e da verificação individual dos cinco arquivos .result dos testes de Alarm Clock. Cada um dos cinco resultados verificados retornou PASS. Também foi analisado o código de timer_sleep() e timer_interrupt() para conferir se o mecanismo implementado corresponde ao comportamento esperado.

8. Reflexão Final

A principal decisão técnica foi substituir a espera ocupada por um mecanismo de bloqueio, permitindo que uma thread permaneça fora da execução enquanto aguarda seu tempo de despertar. A utilização de uma lista específica para threads dormindo facilitou o controle de múltiplas threads com diferentes tempos de espera. Durante a validação, a execução de make check foi interrompida durante um teste de MLFQS, o que mostrou a importância de separar os testes diretamente relacionados à atividade dos demais testes do Pintos.

Também houve uma tentativa inicial de acessar os arquivos de teste pelo caminho incorreto, corrigida com a localização dos arquivos por meio de find. Com mais tempo, seria interessante ampliar a validação com mais testes e investigar separadamente os testes de escalonamento que não fazem parte do escopo principal da atividade.

9. Checklist Final de Entrega

Item	Confirmado?
Compilei do zero seguindo só o que está no relatório	sim
Rodei os testes e evidencias.log foi gerado	sim
Tenho prints obrigatórios	sim
Testei pelo menos um caso limite e um caso inválido	sim
Preenchi a seção de uso de IA	sim
Revisei o relatório e removi frases genéricas / vazias	sim

10. Se eu tivesse mais 2 horas

Eu utilizaria o tempo adicional para ampliar a validação dos testes e garantir uma reprodução mais completa da atividade. Primeiro, executaria uma compilação limpa com make clean seguida de make, verificando se não existem warnings relevantes. Depois, executaria novamente os cinco testes específicos do Alarm Clock individualmente e registraria seus resultados no log. Também investigaria separadamente a execução interrompida de make check, distinguindo os testes relacionados ao Alarm Clock daqueles relacionados a MLFQS e escalonamento. Por fim, revisaria a organização do código para verificar possíveis melhorias na manipulação da lista de threads dormindo e faria uma última execução completa antes de gerar o .tar definitivo.